This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

Scaling Blockchain via Dynamic Sharding

Jianting Zhang, Zicong Hong, Xiaoyu Qiu, Wuhui Chen, *Member, IEEE*, Yufeng Zhan and Song Guo, *Fellow, IEEE*

Abstract—Sharding is considered a promising solution for scaling blockchain systems. However, most existing sharding systems have not considered the dynamics of the environment when making a sharding strategy, including the change of pending transactions, the leaving and joining of participants, and malicious attacks, which could cause performance instability and security issues. To address it, in this paper, we propose an intelligent and efficient dynamic sharding technology to advance the blockchain system performance and security. We first propose a formal and general evaluation framework for blockchain sharding in a dynamic environment, and conclude an optimization target for the system performance and security. To achieve a long-term benefit for the optimization target, a deep reinforcement learning (DRL)-based sharding approach has been proposed to intelligently make optimal sharding strategies. Next, we propose an adaptive resharding protocol to efficiently reduce the overhead introduced by dynamic sharding. Our experimental results illustrate that our proposed dynamic sharding in a simulation testbed can achieve 2.8 times transactions per second compared to traditional static sharding systems, and guarantee high security in a dynamic environment.

Index Terms—blockchain, sharding, scalability, security, performance, deep reinforcement learning

1 Introduction

Blockchain is a decentralized technology, which enables nodes that do not accredit each other to maintain a constantly updated ledger via the consensus protocol [2], [3], [4]. The applications of blockchain range from cryptocurrencies such as Bitcoin [5] and Ethereum [6], to other fields such as data management [7], [8], internet of things [9], [10], and smart grid [11], [12]. However, traditional blockchain protocols cannot meet the requirements of different services due to their poor scalability, e.g., 7 transactions per second (TPS) in Bitcoin and 10 TPS in Ethereum [13].

Sharding is considered one of the most promising solutions to scaling blockchain [13], [14], [15]. The idea of sharding is to divide the system into smaller groups called *shards*, so that each shard can handle transactions in parallel, as well as reducing the overhead of storage, computation, and communication. Nodes (also called *validators*) of the system only validate transactions for one shard, and hence, the scalability of the blockchain is enhanced. To prevent the system from being controlled by the adversary via the adaptive attack [16], [17] and the join-leave attack [18], current sharding systems [19], [20], [21], [22], [23] have to be resharded periodically by reshuffling a fraction of validators or re-electing all shards.

However, all of these sharding systems take a *static sharding strategy* (e.g., the static resharding interval, the static number of shards, the static block size, and the static joining difficulty) without considering the *dynamics* in practical blockchain systems, including: 1) the change of pending transactions because of the dynamic demand in the system; 2) the change of consensus participants due to the leaving and joining of validators; 3) the dynamic malicious attacks due to the change of corrupting ability. The dynamics of blockchain systems can influence the system performance and security. Fig. 1 gives an intuitive example to illustrate the dilemma due to the change of participants and malicious attacks. On the one hand, with more and more new nodes joining a shard (i.e., shard 1 visualized with blue color), the shard size becomes larger, leading to higher overheads of storage, computation, and communication that eventually lower the system performance of sharding. On the other hand, with old nodes leaving or attackers continuously corrupting, the number of honest nodes in shard 2 (i.e., visualized with green color) becomes smaller, leading to the adversary taking over shards more easily. Multiple nodes joining suggests that the system should create more shards to avoid large shard size, while the decrease number of honest nodes in a shard requests that the system should shorten the resharding interval to protect the shard from corruption, or decrease the number of shards to make the shard size large enough to resist attacks. All of them need the system to adjust the sharding strategy dynami-

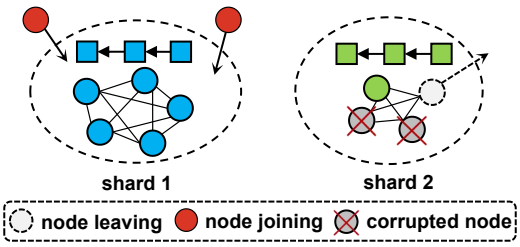


Fig. 1: An example of the dynamics in blockchain systems. The performance of shard 1 (visualized with blue color) will decrease because more new nodes join in the shard, leading to higher overhead of consensus. The shard 2 (visualized with green color) will be easier to attack due to the decreased number of honest nodes.

A short version [1] of this paper is published in ICPP'20.
J. Zhang, X. Qiu and W. Chen are with Sun Yat-sen University, Guangzhou, China. E-mail: zhangjt26@mail2.sysu.edu.cn, qiuxy23@mail2.sysu.edu.cn, chenwuh@mail.sysu.edu.cn.
Z. Hong, Y. Zhan and S. Guo are with The Hong Kong Polytechnic University, Hong Kong, China. E-mail: zicong.hong@connect.polyu.hk, zhanyf1989@gmail.com, song.guo@polyu.edu.hk.
Z. Hong is the corresponding author.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

cally to meet the influence introduced by the dynamics of blockchain systems. Therefore, when most of the sharding systems adopt a static sharding strategy, they cannot meet the balance problem between performance and security under the dynamic blockchain environment.

Considering the influence of dynamics in blockchain systems, in this work, we propose a dynamic sharding technology, the main idea of which is to achieve dynamic sharding intelligently and efficiently when the environment of a sharding system changes.

First, to maintain a long-term benefit of the system performance and security in a complicated and unaware blockchain environment, we propose a deep reinforcement learning (DRL)-based intelligent sharding approach. We analyze the working process of a common sharding system, and define four basic properties that can be used to determine whether a sharding system is practical, including resharding latency, parallelism, shard safety, and shard liveness. Based on these fundamental properties, we obtain an optimization target that takes into account both the system performance and security. The DRL agent has been trained to dynamically select appropriate sharding strategies considering the dynamic environment of sharding systems, for a long-term benefit for performance and security. Then, the trained DRL algorithm is applied in the sharding system via a distributed mechanism to achieve intelligent dynamic sharding.

Next, to efficiently reduce overall overhead introduced by the synchronization and communication needs of the dynamic sharding strategy, we introduce an *adaptive resharding protocol*, which can be divided into three periods: state update, shards adjustment, and shards reorganization. In the period of state update, *state blocks* are designed to reduce storage overhead and accelerate the synchronization of ledgers. Then, a full binary tree structure is proposed to distribute validators into different shards efficiently during the period of shards adjustment. In the shards reorganization period, we adopt an efficient algorithm dominated by shards' leaders to reduce the latency of connection establishment. We finally give a security analysis of our proposed dynamic sharding technology, and show that it can resist many attack models when some system parameters are met.

We sum up the contributions as follows.

- We illustrate the trade-off between system performance and security of sharding systems. We propose a formal evaluation framework where performance and security are formalized by some definitions of basic properties that a practical blockchain sharding system should satisfy.
- We introduce a DRL-based dynamic sharding framework to adjust the sharding strategy dynamically, in order to attain a long-term trade-off between system performance and security in the dynamic environment.
- We introduce an adaptive resharding protocol that helps reshard the system efficiently.
- Experimental results show that the DRL-based sharding framework outperforms other algorithms in maintaining a long-term trade-off between system performance and security.

The rest of this paper is organized as follows. Section 2 introduces related work on blockchain sharding. Section 3 presents the architecture of dynamic sharding. In Section 4, we propose a formal evaluation framework to estimate the performance and security of sharding systems. Section 5 presents our proposed DRL-based dynamic sharding framework. The adaptive resharding protocol is presented in Section 6. In Section 7, we give a security analysis. Section 8 contains experimental results. Finally, Section 9 gives the conclusion and discussion.

2 Related Work

A sharding system is processed in time intervals, called *epochs*. Each epoch includes two stages: consensus stage and resharding stage. The consensus stage is responsible for processing transactions, while the resharding stage enhances the security of the system. According to [27], there are two main types of sharding in blockchain: partial sharding and complete sharding.

2.1 Partial Sharding Systems

To address the scalability in non-sharding blockchain systems, partial sharding has been proposed.

Elastico [19] is the first well-known partial sharding system, and Zilliqa [24] is another well-known partial sharding system that extended the idea of Elastico to support smart contract execution. In these partial sharding systems, each node needs to maintain the whole ledger, and each committee is responsible to consensus on a disjoint subset of transactions via Practical Byzantine Fault Tolerance (PBFT) [28] protocol. At each epoch, a final committee packs all transactions received from committees into a final block and then broadcasts the new block to each node for global ledger update.

Since both Elastico and Zilliqa are permissionless blockchains and adopt PBFT as their intra-committee consensus protocol, an attacker can present multiple identities to control these PBFT-based consensus committees, also called the *Sybil* attack [18]. To prevent the Sybil attack, Elastico and Zilliqa adopt a Proof of Work (PoW)-based sybil-resistance scheme, where every system node needs to compute a PoW puzzle to acquire an identity at the beginning of each epoch. Although these partial sharding systems achieve sharding for transaction computation, they do not partition the ledger, and still suffer from heavy storage overhead.

2.2 Complete Sharding Systems

To further lower the overload of storage, many works propose complete sharding, i.e., sharding for transaction computation, network communication, and ledger storage.

OmniLedger [21] is the first well-known complete sharding system, where the entire storage is divided into several subsets that are equal to the number of shards and thus reduces the cost of storage. To prevent the adaptive attack [29] that the adversary can control a part of honest nodes in a single shard and the join-leave attack [30] where the adversary rejoins a specific shard using fresh identities, OmniLedger swaps some nodes among different shards per

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

System	Model	Consensus	Storage per Node	Scalability	Resharding Interval	Sharding Strategy
Elastico [19]	UTXO	PoW + BFT	$O(X)$	partial	1 block interval	static
OmniLedger [21]	UTXO	PoW + BFT	$O(X/k)$	complete	1 day	static
RapidChain [22]	UTXO	PoW + BFT	$O(X/k)$	complete	1 day	static
Zilliqa [24]	Account	PoW + BFT	$O(X)$	partial	1 block interval	static
AHL [25]	Account	TEE + BFT	$O(X/k)$	complete	fixed(not provided)	static
SSHC [26]	UTXO	PoW + BFT	$O(X/k)$	complete	bounded	static
Ours	Account	PoW + BFT	$O(X/k)$	complete	**adaptive**	**dynamic**

TABLE 1: Comparison of our sharding system with state-of-the-art blockchain sharding systems (with n system nodes, k shards, X total storage)

epoch (a fixed time, i.e., one day). However, this resharding mechanism needs to swap up to 2/3 of the shard's size to guarantee the security of the system, introducing a huge synchronization overhead for a swapped node.

To enhance the performance and security of the resharding stage, RapidChain [22] proposes the bounded Cuckoo rule [31], where a smaller number of nodes are reshuffled every epoch, ensuring a bounded deviation in the sizes of the committees and lowering the resharding overhead. AHL [25] introduces the Trust Execution Environment(TEE) to help generate epoch randomness and verify nodes' identities, enhancing the security of the resharding stage. SSHC [26] adopts a sequential committee selection where shards are formed one by one, to prevent an adversary from focusing on attacking a specific shard.

However, the experiments of above sharding systems are typically simulated under their own ideal models without incorporating *dynamic behaviors* (including the increase/decrease of pending transactions with the increase/decrease requirement of nodes, the increasing number of whole system nodes with new nodes joining, the decreasing amount of nodes in a shard due to arbitrarily leaving of old nodes and the continuous corrupting by malicious nodes). The dynamic behaviors are common in a realistic blockchain system and can break the trade-off between the system performance and security. For example, to reduce the overhead introduced by frequent reshardings, the system can prolong the resharding interval. However, once a mass of nodes leave a shard suddenly (i.e., the dynamic leaving of nodes reduces the size of the shard), a longer resharding interval will make the adversary take over the shard more easily by corrupting enough honest validators of a shard.

Table 1 shows a comparison between our work and previous sharding systems. In summary, the existing sharding systems can boost the scalability of blockchain systems, but they fail to attain a long-term trade-off between system performance and security when the environment of sharding systems continuously changes. The dynamic of the blockchain environment motivates us to design a dynamic sharding system to maintain a long-term high performance and security of blockchain systems.

2.3 Dynamic Sharding

To adapt to the dynamic feature in practice, dynamic sharding protocols have been proposed [32], [33], [34], [35], [36], [37]. In particular, some prior works [32], [33], [34], [35] propose dynamic data management/partition mechanisms to reduce the overhead of cross-shard transactions. The high-level idea of these protocols is to dynamically mitigate the accounts among distinct shards to minimize the coordination of handling cross-shard transactions, which is an optimization of the application layer since it does not change the underlying consensus groups and resharding mechanisms. Our protocol, instead, is a dynamic sharding protocol that optimizes the performance and security from the aspect of the system layer, which dynamically adjusts consensus groups and resharding mechanisms to prevent potential attacks while maintaining optimal performance. It is worth noting that our dynamic sharding protocol is compatible with the above dynamic data management/partition mechanisms. SecuSca [36] proposes a dynamic sharding approach to balance security and scalability, but fails to guarantee the long-term trade-off between them due to its greedy algorithm. DSPO-CB [37] presents a DRL-based sharding model to dynamically assign nodes into shards. However, DSPO-CB focuses on permissioned blockchains, while our protocol can work for permissionless blockchains via a hybrid consensus. This makes our protocol more general and practical.

3 Dynamic Blockchain Sharding Architecture

3.1 System and Network Model

Our system model focuses on permissionless blockchains. As most previous works do [19], [21], [22], [26], we adopt a *hybrid consensus* [16] to support our sharding system, i.e., combining an inefficient blockchain protocol for bootstrapping the permissionless system with a fast permissioned consensus protocol. Specifically, we use the PoW protocol to establish nodes' identities, protecting against Sybil attacks by requiring every node that wants to join or stay in the system to solve a PoW puzzle. In such a *random shard-assignment* mechanism, new nodes will be assigned to shards based on their randomly obtained identities. Then, a more efficient BFT protocol is used for the intra-shard consensus to create blocks within shards. Besides, similar to [21], [38], [39], there is a single shard (called *identity shard*[1]) coordinating the shard reconfiguration and maintaining an *identity chain* that is responsible for the registry of participants in our system.

Like most sharding systems [21], [39], [40], [41], [42], we assume the underlying network is partially synchronous, where there is an unknown global stabilization time (GST) after which all message delays between honest nodes are bounded by a known Δ.

3.2 Security Assumptions and Threat Model

As prior work [19], [21], [22], [23], [25], we assume that there are at most sn malicious/Byzantine nodes in our sharding

1. It is called ordering shard in [39] and called control shard in [38].

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

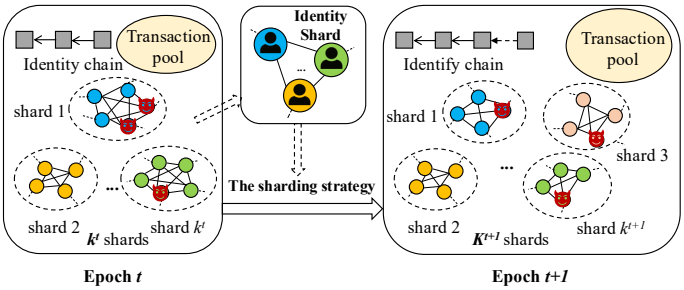


Fig. 2: System overview

system all the time, where n is the number of system nodes and s is called the system fault tolerance. Similarly, we define the shard fault tolerance as f, which indicates a shard with m nodes can tolerate at most mf malicious nodes. We assume that Byzantine nodes in our system can behave arbitrarily, but are computationally bounded and cannot break the cryptographic primitives.

3.3 System Overview

Our dynamic sharding system proceeds with epochs, each of which contains two stages: consensus stage and resharding stage.

A consensus stage includes multiple consensus rounds/instances, and a new block will be created in each shard in each consensus round via a PBFT-like consensus protocol. Besides, every node that wants to join or stay in the system is required to solve a PoW puzzle during this stage. For a new node, the PoW solution is used for its identity in the system. For an old node that has obtained the identity in a previous epoch, the PoW solution is used to ensure its legality so that the adversary cannot continuously add new nodes to the system. Note that the PoW puzzle is generated based on the epoch randomness and can be solved offline, so the adversary cannot pre-compute the solutions, and the system can keep on handling transactions without waiting for the solutions.

A resharding stage following each consensus stage is introduced to enhance the system security, where the PoW solutions will be verified, and the system will readjust shards based on a new sharding strategy that is selected by our proposed DRL-based dynamic sharding algorithm. To prevent attacks that malicious nodes strategically leave and rejoin the system or periodically corrupt honest nodes, a subset of shard nodes is switched between different shards as in [21], [22].

An example of the transition process of our system is illustrated in Fig. 2. At the end of epoch t, the identity shard (visualized through the different colors) agrees on a new identity block that contains a sharding strategy selected based on the environment of epoch t. Then, the system readjusts current shards according to the new sharding strategy and enters the next epoch $t+1$ to continue processing transactions, i.e., the number of shards changes from k^t to k^{t+1}. Next, we introduce three main components of our dynamic sharding system.

Evaluation framework for sharding in a dynamic environment. Four properties have been proposed to determine whether a sharding system is practical or not, including *ressharding latency*, *parallelism*, *shard safety*, and *shard liveness*. The first two properties are used to evaluate the system performance, while the last two properties are used for the system security. Intuitively, a practical sharding system should be high-performance and high-security. The evaluation framework is described in Section 4.

DRL-based dynamic sharding algorithm. Some factors, including the number of consensus rounds, the number of shards, the block size, and the difficulty of PoW, have been extracted from the evaluation framework as the sharding strategy. A DRL-based sharding algorithm is designed to help select optimal sharding strategies to achieve a long-term trade-off between system performance and security. More details of the DRL-based sharding algorithm are described in Section 5.

Adaptive resharding protocol. An adaptive resharding protocol is designed to reduce the latency introduced by the resharding phase, which consists of three periods: state update, shards adjustment, and shards reorganization. We will outline the protocol in Section 6.

4 Evaluation Framework for Blockchain Sharding System in Dynamic Environment

In this section, we first give the formal definitions of a sharding system. Then we analyze the unique operational process of sharding systems, and conclude four general properties used to evaluate the performance and security of sharding systems.

4.1 Definitions of Blockchain Sharding System

A sharding system containing k^t shards at epoch t can be denoted as a collection of sets $S^t = \{S_1^t, S_2^t, ..., S_{k^t}^t\}$, where $S_i^t (1 \leq i \leq |k^t|)$ represents a transaction-processing shard (i.e., committee), which is responsible for the verification and process of parts of transactions. Let $n^t = \sum_{c=1}^{k^t} |S_c^t|$ denote the total number of nodes (validators in all shards) and $m^t = \frac{n^t}{k^t}$ represent the average number of nodes per shard (i.e., shard size) at epoch t. To better illustrate the working process of blockchain sharding systems, we first describe two critical components in sharding systems:

1) **Intra-shard consensus** is used to reach an agreement on a new block during the consensus stage. The latency of the intra-shard consensus is used to evaluate the block interval from block generation to block commitment by the entire network. Most sharding systems [19], [21], [22] adopt the PBFT-like protocol as their intra-shard consensus. Therefore, we follow previous work [43], [44], [45], [46] to quantify the consensus latency. A round of PBFT consensus consists of three main phases, i.e., pre-prepare T_{prep}, prepare T_{pre}, and commit T_{com}. In each phase, every node will forward, verify, or record block data. Finally, the time cost of the round T_{intra} is proportional to shard size m^t and can be measured by

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

$$\begin{aligned} T_{intra} =& T_{prep} + T_{pre} + T_{com} \\ =& (\frac{S^B log m^t}{R_t} + t_v) + (\frac{S^H log m^t}{R_t} + t_v) \\ &+ (\frac{S^H log m^t}{R_t} + t_v + t_a) \\ =& 3t_v + t_a + \frac{S^B log m^t}{R_t} + \frac{2S^H log m^t}{R_t}, \end{aligned} \tag{1}$$

where t_v is the cost to validate the message of each phase, t_a is the cost for a new block appended to the blockchain, S^B is the size of a block, S^H is the size of a block header, and R_t is the transmission rate of data. Note that a completely new block is only broadcast once, e.g., the pre-prepare phase broadcasts the whole block, while the next two phases only broadcast the block header. Besides, messages are delivered by all nodes in $O(log m^t)$ times at most (like [22], [47], we do not assume all-to-all communication for efficiency).

2) **Cross-shard transactions** is formed due to the separated storage of ledgers [48], [49]. Those transactions whose senders and receivers are recorded in different shards are called cross-shard transactions. Cross-shard transactions induce related shards to communicate with each other. To ensure the atomicity, a cross-shard transaction needs to be split into several transactions, called redundancy transactions or relay transactions, which are used as receipts to confirm the correctness of the original transaction in the sender's shard. Therefore, the system needs to handle several relay transactions when handling a cross-shard transaction. Let R_n be the average amount of relay transactions required by a cross-shard transaction, and P_c be the probability of cross-shard transactions in the sharding system. We can calculate the rate of *practical transaction* P_r by

$$P_r = \frac{1}{1 \times (1 - P_c) + (1 + R_n) \times P_c} = \frac{1}{1 + R_n P_c}. \tag{2}$$

The rate P_r is used to compute the practical number of transactions, i.e., the transactions other than relay transactions. For example, in Monoxide [23], where $R_n = 1$ and $P_c = 1 - \frac{1}{k^t}$, if the system has processed 1000 transactions with 3 shards, the practical number of transactions being processed is $1000 \times P_r = 1000 \times \frac{1}{1+1\times(1-\frac{1}{3})} = 600$, and the number of relay transaction is 400.

4.2 Performance Evaluation

We measure the system performance with transaction throughput, i.e., the number of transactions that the system can process per second. Compared to classic non-sharding blockchain systems, sharding systems have two features when handling transactions. The one is sharding systems introduce a resharding (or reconfiguration) stage, whose overheads should be evaluated. The other is sharding systems support to handle transactions in parallel, which enhances transaction throughput significantly and deserves to be quantified in detail. For this end, we respectively define *resharding latency* and *parallelism* to measure the performance of sharding systems.

Definition 1 (Resharding latency)**.** *Resharding latency indicates how long a sharding system takes to reconfigure shards, represented by* T_{res}.

Note that different blockchain sharding systems adopt different resharding mechanisms, leading to various resharding latencies. In this paper, we define an evaluation for resharding latency for our dynamic sharding protocol by following [43], which can be measured in three stages:

1) Shards update their ledgers respectively. The latency of this stage is denoted as T_s.
2) A new identity block, which includes epoch randomness, new nodes' identities, and the new sharding strategy, will be determined in this stage. This stage is denoted as T_i.
3) Nodes spontaneously form predetermined shards for the next epoch based on the new identity block. This part is denoted as T_a.

At the first stage, each shard creates a special block, called *state block* and denoted as sb, whose responsibility is to record the latest information of all accounts of a shard's ledger. Specifically, at the start of the resharding stage of epoch t, each leader of shards traces all valid transaction blocks since the state block sb^{t-1} at epoch $t-1$. Then the leader gathers the balance of all accounts and writes this information into the body of a new state block sb^t, which will be broadcast to other nodes to reach a consensus. The main activity of the update stage is to reach a round of intra-shard consensus, and thus we can make $T_s = T_{intra}$.

At the second stage, a new epoch randomness is constructed. Since epoch randomness generation is not the research focus of this paper, we assume that there is an efficient randomness beacon protocol (such as [50], [51], [52]) to generate an unbiased epoch randomness by the identity shard. For simplicity, we assume the latency of the randomness generation stage is a constant κ. Then, a global leader elected with epoch randomness verifies new nodes' identities, selects a new sharding strategy, and packs this information into a new identity block. After that, the global leader coordinates to run an instance of PBFT-like consensus to append the identity block to the identity chain. Therefore, the latency of this stage consists of the creation of epoch randomness and the identity block, i.e., $T_i = T_{intra} + \kappa$.

At the third stage, shard nodes are forced to adjust based on the new identity block. Specifically, some shards may be merged or split, causing relevant shard nodes to re-establish communications with other nodes and synchronize necessary ledger states so that they can properly handle transactions next epoch. Obviously, the greater the number of shards to be adjusted, the greater the time overhead. We use $|k^t - k^{t-1}|$ as the independent variable to represent the changing number of shards, and introduce an evaluation factor α to evaluate the time overhead of shards merge or split. Besides, recall from Section 3.3, the sharding system will switch a subset of nodes between different shards, where a switched node also needs to re-establish communications with other nodes and synchronize the ledger state of its new shard. We denote this overhead for a node switching shards as β. Note that switching nodes happens after the shards adjustment to prevent switched nodes from remaining connected to the original shard nodes. Therefore,

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

the time overhead of this stage can be calculated by

$$T_a = \alpha|k^t - k^{t-1}| + \beta. \tag{3}$$

Finally, we can conclude the resharding latency T_{res}^t at epoch t as

$$\begin{aligned} T_{res}^t &= T_s + T_i + T_a \\ &= 2T_{intra} + \kappa + \alpha|k^t - k^{t-1}| + \beta. \end{aligned} \tag{4}$$

Definition 2 (Parallelism)**.** *Parameterized by k^t (the number of shards) and P_r (the practical transaction rate), it indicates the capability of paralleling processing practical transactions.*

Sharding technology enhances transaction throughput significantly by dividing the whole system into multiple shards so that transactions can be processed in parallel. The parallelism is defined to evaluate how much improvement in transaction throughput a sharding system can achieve compared to classic non-sharding systems. Ideally, a fully sharding system with k^t shards can reach k^t times throughput than a non-sharding system. However, some factors limit the sharding system from reaching this goal in practical scenarios. On the one hand, the sharding system introduces resharding overheads that take up the time used to process transactions. On the other hand, the system needs to handle relay transactions introduced by cross-shard transactions (Section 4.1), leading to the practical transaction throughput being less than the computed throughput. As a result, the parallelism should be reflected by the *practical capability* of processing transactions, i.e., how many practical transactions a system can process per second.

To measure the practical transaction throughput of a sharding system, we first define the time of the consensus stage at epoch t as T_e^t. As Section 3 describes, each epoch includes two stages: consensus stage and resharding stage. We define that each consensus stage contains r^t consensus rounds, in which shard S_i^t creates one new transaction block within latency $T_{intra,i}$. Then we can easily get T_e^t by

$$T_e^t = \max_{1 \le i \le |k^t|} r^t T_{intra,i} + T_{res}^t. \tag{5}$$

Furthermore, the number of transactions Ω_i^t that shard S_i^t can process during epoch t can be calculated by

$$\Omega_i^t = \frac{(S^B - S^H)r^t}{S^T}, \tag{6}$$

where S^T denotes the average size of one transaction. Next, the entire number of transactions Ω^t processed by a sharding system during epoch t is

$$\Omega^t = \sum_{i=1}^{k^t} \Omega_i^t = \frac{(S^B - S^H)r^t k^t}{S^T}. \tag{7}$$

Considering the influence of cross-shard transactions as described above, we finally measure the system performance of a sharding system by computing the practical transaction throughput O:

$$O = \frac{\Omega^t P_r}{T_e^t} = \frac{(S^B - S^H)r^t k^t}{S^T T_e^t (1 + R_n P_c)}. \tag{8}$$

4.3 Security Evaluation

Although sharding technology enhances transaction throughput, it also introduces some new security challenges due to the decreased size of a consensus group. As discussed in Section 2, the adversary can continuously corrupt honest nodes of a shard, or selectively generate node identities that belong to the same shard. These malicious behaviors will enable the adversary to control the shard, and thus threaten the system security. In this paper, we comprehensively analyze the whole operational process of a sharding system, and systematically quantify the system security, which contains several attacks analyzed by prior works and some new security properties that have not yet been considered. We respectively define shard safety and shard liveness to evaluate the security of sharding systems. Note that the security of the entire blockchain/system can be inherently guaranteed during the consensus stage if each shard satisfies the shard safety and shard liveness. Briefly, secure shards are responsive (i.e., live) and ensure that both intra-shard and cross-shard transactions can be handled by relevant nodes consistently (i.e., safely). Many recent works have proved this [39], [41], [42], [53], and thus we omit the rigorous proof in this paper.

Definition 3 (Shard safety)**.** *Shard safety indicates that a shard is not controlled by malicious nodes during the shard formation process and the shard consensus process, parameterized by λ (shard formation security parameter), ϵ (the average attack ability of a dishonest node) and $\varkappa$ (the participating constraint factor).*

Intuitively, shard safety is a prerequisite for system security because if any shard is controlled by the adversary, other shards will interact with invalid data provided by the malicious shard via cross-shard transactions. According to the operational process of sharding systems, we conclude three attacks that exclusively exist in sharding systems.

1) **Unsafe shard formation attack**. The adversary can control a specific shard during the process of shard formation, called the *unsafe shard formation attack*. Specifically, once the number of malicious nodes that are divided into the same shard is more than the shard fault tolerance (e.g., $1/3$ in classic PBFT protocol), the consensus process of this shard is manipulated by malicious nodes.

Although nodes are randomly selected to form shards, this attack can still be launched with a high possibility if the shard size is too small [19]. As in previous sharding systems [22], [25], the probability of forming an unsafe shard in a random way can be modeled as a hypergeometric distribution. We define X as a random variable that serves as the number of dishonest nodes assigned to the same shard. Let f denote the fault tolerance of a shard, which represents that an honest shard can tolerate at most $m^t f$ dishonest nodes. Once the number of dishonest nodes in a shard exceeds $m^t f$, the shard can be deemed to be unsafe. For example, in a PBFT-based consensus protocol, the consensus group can tolerate malicious nodes at most $1/3$ of the total nodes. Similarly, we set $F = sn^t$ as the number of the whole malicious nodes in a sharding system with n^t nodes and the fault tolerance of the system s. The s has a flexible value that is determined by the security model of a sharding system, e.g., $s = 1/4$ in OmniLedger [21], $s = 1/3$ in RapidChain

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

[22], and AHL [25]. Finally, the probability for forming an unsafe system is:

$$P[X \geq \lceil m^t f \rceil] = \sum_{x=\lceil m^t f \rceil}^{m^t} \frac{\binom{F}{x}\binom{n^t - F}{m^t - x}}{\binom{n^t}{m^t}}. \quad (9)$$

Similar to previous works [21], [22], a parameter λ is used as a constraint to ensure the security of forming shards. We consider it safe enough if the following formula is met.

$$P[X \geq \lceil m^t f \rceil] < 2^{-\lambda}. \quad (10)$$

2) **Corruption attack**. The adversary can control a shard during the consensus stage each epoch, called the *corruption attack*. The corruption attack has been studied in previous works [16], [54], [55] and is assumed to be widespread in existing sharding systems [19], [21], [22], [25], where the adversary can adaptively corrupt nodes to eventually control the whole consensus group.

However, in a real scenario, corrupting a node is typically not instant since it takes a while to infect an otherwise clean host [16]. We therefore define a corruption attack model where a malicious node is with average attack power ϵ^t. The ϵ^t indicates the number of honest nodes that the malicious node can corrupt successfully per unit time (e.g., second). Prior sharding systems introduce the resharding stage to periodically switch a subset of shard nodes to revive the corrupted nodes. As a result, the adversary can only target corrupting nodes in the consensus stage each epoch. With the assumed nodes number n^t, the system fault tolerance s, and the duration of the consensus stage $(T_e^t - T_{res}^t)$, we can quantify the number of corrupted nodes in a shard at epoch t as: $\epsilon^t n^t s(T_e^t - T_{res}^t)$.

To guarantee that a shard is not completely corrupted by the adversary, the number of corrupted nodes should always be lower than the maximum Byzantine nodes the shard can tolerate. Finally, we can conclude the following security constraint:

$$\epsilon^t n^t s(T_e^t - T_{res}^t) \leq m^t f. \quad (11)$$

The right of Eq. (11) represents the maximum Byzantine nodes a shard can tolerate, where m^t is the shard size and f is the shard fault tolerance. This formula requests that all attackers of a system cannot completely corrupt any one of the safe shards before the sharding system gets into the resharding stage.

3) **Multiple identities attack.** The adversary can control a specific shard by selectively creating multiple nodes' identities that belong to the shard, called the *multiple identities attack*. Since a sharding system adopts the PoW protocol to establish identities and distributes nodes into different shards based on their PoW solutions, a node with high computing power may obtain multiple identities by solving several PoW puzzles in an epoch. Therefore, a malicious node with computing power that is higher than others can obtain massive identities during an epoch and select a specific shard to attack. Previous sharding systems set the difficulty of PoW puzzles used for establishing identities as fixed. For example, RapidChain [22] assumes each node has a fixed and identical computing power, and sets a fixed difficulty of PoW puzzles by which each node can only solve one puzzle during each 10-minute period. Such assumptions, however, are not realistic in a practical network where nodes have dynamic computing power. To prevent the multiple identities attack, we limit the number of puzzle solutions that a node can obtain at epoch t by:

$$\frac{p_c^t}{p_d^t} \leq \varkappa k^t, \quad (12)$$

where p_c^t is the average computing power of the adversary, p_d^t presents the PoW difficulty for nodes joining at epoch t and $\varkappa \leq 1$ is a constraint factor. More specifically, $\frac{p_c^t}{p_d^t}$ indicates the times that a malicious node ultimately computes the PoW solutions. The constraint factor for a multiple identities attack limits the difficulty for the adversary to attack shards.

Definition 4 (Shard liveness)**.** *Shard liveness indicates that any transactions are ultimately accepted by all honest nodes if the transactions are valid, parameterized by η (the view-change rounds).*

In a shard, a malicious node could be selected as the leader of the consensus group, and then hinders the consensus process by performing inconsistent behaviors, making a block pending and not being committed to the blockchain even if the block is valid. To protect against such behaviors and ensure the liveness of this consensus group, the PBFT consensus protocol designs the view-change mechanism [56]. During the view-change phase, a new leader is selected to take over the consensus process. Other non-leader nodes send the new leader with their view-change requests that contain information about the pending block and the local latest states (also called *views*). The new leader will decide to re-commit the pending block according to the quorum of received requests (e.g., $f + 1$ same views). By re-selecting a new leader, the PBFT protocol ensures liveness in classic non-sharding blockchain systems.

However, sharding technology introduces new liveness problems because of the system resharding. Specifically, on the one hand, since the sharding systems need to switch a subset of nodes among different shards at each epoch, a new leader may not receive sufficient view-change requests. Thus, the new leader cannot decide which view should be used, and the pending block cannot be re-committed, compromising the liveness. On the other hand, if shard ledgers are adjusted next epoch (e.g., a shard will be divided into two shards, generating two new ledgers), the transactions in the pending block may no longer be handled by only one shard, since users of these transactions will be maintained by different shards in the next epoch and these transactions should also be sent to different shards for processing. As a result, the pending block will be abandoned because it contains transactions that are from different shards. To avoid the above situations and ensure the *shard liveness*, a new mechanism should be designed.

We quantify such a shard liveness by using the probability of selecting at least one honest node as the shard leader by view changes, called *liveness probability*. Intuitively, the higher the liveness probability is, the easier the shard liveness can be satisfied, since honest leaders will recommit pending blocks and make every honest node reach

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

a consistent and valid ledger state. To compute the liveness probability, we define a variable η to represent that there are η consecutive rounds selecting malicious nodes as the shard leader while the $(\eta+1)$th round is with an honest leader. Then, to ensure a high liveness probability, the following formula should be satisfied:

$$P[\eta] = \prod_{i=0}^{i=\eta-1} \frac{m^t s - i}{m^t - i} \cdot \frac{m^t - m^t s}{m^t - \eta} < s^\eta \cdot 1 < 10^{-5}, \quad (13)$$

where $m^t s$ is the average number of malicious nodes in a shard with the system fault tolerance s. This formulation indicates the probability $P[\eta]$, which a shard cannot restore its liveness within consecutive η view-change rounds, should be small enough, i.e., less than 10^{-5}. On the contrary, in this case, the probability that an honest node is chosen as the shard leader with limited η view changes is large enough. From Eq. (13), we can calculate that $\eta > \log_s 10^{-5}$, where $\log_s 10^{-5}$ is a constant depending on s.

To conclude, when a shard triggers more than $\log_s 10^{-5}$ view changes (i.e., $\eta > \log_s 10^{-5} \approx 8$, with $s = 1/4$), the shard liveness is satisfied with a high probability (i.e., more than $1 - 10^{-5}$). Therefore, in order to set aside enough time for a shard to trigger enough view changes to guarantee the shard liveness, the duration of the consensus stage at each epoch should follow the constraint:

$$(\eta+1)T^t_{intra} \leq T^t_e - T^t_{res}. \quad (14)$$

We assume the overhead of a round of the view change is approximately equal to that of the intra-shard consensus. Therefore, the left part of Eq. (14) represents the overhead of triggering $(\eta+1)$ view changes. A sharding system can ensure the shard liveness if the constraint Eq. (14) is met.

5 DRL-based Dynamic Sharding Framework

5.1 Problem Statement

Section 4 respectively concludes the quantitative formulas for the system performance and security of sharding systems. Note that the definitions of performance evaluations are the optimistic metrics with the assumption that the system satisfies the security properties. However, from Eq. (1) to (14), we can find that the system performance and security in blockchain sharding are a pair of contradictory objectives in an optimization problem.

On the one hand, raising the number of consensus rounds means that there is more time for shards to process transactions within a period of time (Eq. (5)), and expanding the amount of shards means that there are more groups to handle transactions in parallel (Eq. (7)), and enlarging the block size enables more transactions to be processed in a block (Eq. (6)). All of these settings can increase transaction throughput (Eq. (8)), and thus enhance the system performance of sharding systems. However, increasing the number of consensus rounds r^t, shards number k^t, and the block size S^B will decrease the security. Specifically, a larger k^t leads to a smaller shard size m^t when the overall number of nodes n^t is determined, making it easier for the attackers to form an insecure shard or take over a shard (Eq. (9) to (11)). Similarly, when increasing r^t and S^B, the risk that a shard to be corrupted by the attackers will increase, because there are more time for the adversary to launch the corrupting attack (Eq. (1), Eq. (5) and (11)).

On the other hand, in order to increase the resiliency to the attacks described in Section 4.3 and diminish security risk, a sharding system can enlarge the shard size m^t (i.e., reduce k^t), decrease consensus rounds r^t, or reduce block size S^B. Nevertheless, these measures also lower the transaction throughput since the parallelism of sharding systems will be decreased and the total time for consensus will be shortened (Eq. (5) to Eq. (7)).

Besides, the difficulty of PoW puzzles also impacts both the system performance and security. Specifically, if the PoW difficulty p_d is too low, the malicious nodes can compute several PoW solutions and obtain multiple different identities to select a specific shard to attack, and then the probability for the malicious attackers to form an insecure shard will increase. On the contrary, if the PoW difficulty p_d is too high, honest nodes with limited computing power cannot solve PoW puzzles within a limited time, and thus cannot participate in the consensus of the next epoch. The decreased number of nodes will reduce the system performance. Therefore, the PoW difficulty p_d needs to be adjusted dynamically when the average computing power of the nodes changes.

To get a practical sharding system that satisfies the properties of low resharding latency, high parallelism, shard safety, and shard liveness, we maximize the transaction throughput under the premise of guaranteeing security. Finally, we can conclude the problem formulation as follows.

$$\max \quad O = \frac{(S^B - S^H) r^t k^t}{S^T T^t_e (1 + R_n P_c)}, \quad (15)$$

$$\begin{aligned} s.t. \quad & P[X \geq \lceil mf \rceil] < 2^{-\lambda}, \\ & \epsilon^t n^t s (T^t_e - T^t_{res}) \leq m^t f, \\ & \frac{p^t_c}{p^t_d} \leq \varkappa k^t, \\ & (\eta+1) T^t_{intra} \leq T^t_e - T^t_{res}. \end{aligned} \quad (16)$$

As analyzed above, the consensus rounds r^t, the number of shards k^t, and the block size S^B affect the transaction throughput and the security. Finally, we summarize the factors that impact the system performance and security of sharding as follows: the consensus rounds r^t, the number of shards k^t, the block size S^B, and the PoW difficulty p_d. We define these factors as *the sharding strategy*. Our goal is to make a proper sharding strategy for every epoch to gain a final long-term trade-off between the system performance and security of sharding.

Solving this issue is difficult because of the non-linear constraints and unawareness of the dynamic blockchain environment. Moreover, blockchain is considered a long-running system that must always ensure security. As a result, we need to consider the long-term benefits when making decisions on the sharding strategies. Given the complicated state space and the long-term requirements, our protocol chooses to use deep reinforcement learning (DRL) to solve the problem. DRL can effectively handle sequential decision-making tasks and has been shown to be able to optimize complex systems with large state and action spaces. Our experiments in Section 8 also confirm its effectiveness in solving our problem.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

5.2 DRL Model Design and Training

In DRL, an agent (i.e., the action performer) selects an action based on the current environment state, and then the environment returns a reward based on the selected action. To describe such interactions between the environment and the agent, we define the three critical elements of our DRL-based sharding model: states, actions, and rewards.

5.2.1 States

We define the states based on the variables that affect the selection of sharding strategies, and can be observed or measured in sharding systems, including: the total amount of pending transactions q_{tx}, the total number of nodes n, and the average computing power p_c. The pending transactions are stored in transaction pools of shards and are waiting to be processed by their relevant shards. Similarly, we can get the total number of nodes n from the newest identity block. To determine the average computing power p_c, we adopt the assumption, like Elastico [19], that the network hashrate is used to estimate nodes' computing power. Specifically, the sharding system utilizes the current difficulty of PoW puzzles and relevant timestamps submitted by shard nodes to compute the network hashrate. The hashrate refers to the amount of hashing computation completed by the network per second, and thus can be used to estimate the average computing power of nodes with the given number of nodes. Finally, the environment state at epoch t can be denoted by

$$s_t = [q_{tx}, n, p_c]_t. \tag{17}$$

5.2.2 Actions

In our DRL-based sharding model, the action space is defined with the sharding strategy. As analyzed above, a sharding strategy contains the consensus rounds r, the number of shards k, the block size S^B, and the PoW difficulty p_d. Note that a sharding strategy is made in the resharding stage, and will continue until the sharding system enters the next epoch resharding stage. The action at epoch t can be defined by

$$a_t = [r, k, S^B, p_d]_t. \tag{18}$$

5.2.3 Rewards

The reward function in a DRL model is used to evaluate the sharding strategy made at each epoch. Intuitively, the higher reward means that the better action has been made. The target of a DRL algorithm is to maximize the reward by continuing to train the DRL agent. Since the problem in our sharding system is also an optimization issue whose goal is to maximize the transaction throughput under the premise of guaranteeing system security, we adopt system throughput as the reward function in our DRL-based sharding framework. The following security conditions are used as the constraints of the reward function at epoch t

$$\begin{aligned}
&\Phi_1 : P[X \geq \lceil m^t f \rceil] < 2^{-\lambda}, \\
&\Phi_2 : \epsilon^t n^t s(T_e^t - T_{res}^t) \leq m^t f, \\
&\Phi_3 : \frac{p_c^t}{p_d^t} \leq \varkappa k^t, \\
&\Phi_4 : (\eta + 1) T_{intra}^t \leq T_e^t - T_{res}^t.
\end{aligned} \tag{19}$$

If all of these constraints are satisfied, meaning that the security at epoch t has been guaranteed, the reward is equal to the transaction throughput. Otherwise, the reward is set to be 0 to represent that the current sharding strategy is improper for the present environment state. Ultimately, the reward r_t at epoch t can be formulated by

$$r_t(s_t, a_t) = \begin{cases} 0, \text{ if one of } \Phi_1 - \Phi_4 \text{dissatisfies}, \\ \frac{(S^B - S^H) r^t k^t}{S^T T_e^t (1 + R_n P_c)}, \text{ otherwise.} \end{cases} \tag{20}$$

5.2.4 Training Methodology

Before applying the DRL-based sharding algorithm, we should first train the DRL agent to adapt to the dynamic sharding system. A DRL agent maintains two initial neural networks at the beginning, respectively called actor network θ^μ and critic network θ^Q. The main function of the former is to output a sharding strategy based on the input, i.e., the environment state of the sharding system. The latter is an evaluation of the sharding strategy. The training process is a series of interactions between the environment of the sharding system and the agent. We set T_{step} as a pre-setting value for the number of resharding steps in a training episode. At every training episode, the agent utilizes the actor network to make T_{step} sharding strategies π_{θ^μ}, and then computes the cumulative expected rewards $Q^{\pi_{\theta^\mu}}$ according to the critic network θ^Q. After that, the agent updates the critic network by comparing the expected rewards and the actual rewards of every resharding step (computed by Eq. (20)), and then updates the actor network according to the cumulative expected rewards. The training process will be complete if the cumulative rewards become stable. Note that the training process can be done completely offline and will not introduce overhead when deploying the well-trained DRL agent to make sharding strategies. In the next Section, we will show how the trained agents can be applied to the sharding system.

6 Adaptive Resharding Protocol

In this section, we introduce the adaptive resharding protocol, an efficient and practical ledgers merging or splitting method that adjusts shards every epoch to solve potential problems of dynamic sharding technology. The protocol contains three periods: state update, shards adjustment and shards reorganization.

6.1 State update

Similar to previous sharding system [21], at the start of resharding stage, each shard agrees on a new state block that records the information of its own shard. In contrast to transaction blocks that primarily record the transaction data, state blocks mainly store accounts information. State block methodology is compatible with blockchain dynamic sharding systems. On the one hand, it helps resharding nodes (including new nodes and switching nodes whose shard assignments change at next epoch) rapidly bootstrap and synchronize to the latest state of the shard. Resharding nodes attain the state of a shard by only downloading the newest state block instead of the blockchain history. After that, resharding nodes in a shard can simply verify the validity of transactions whose sender's address is maintained

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

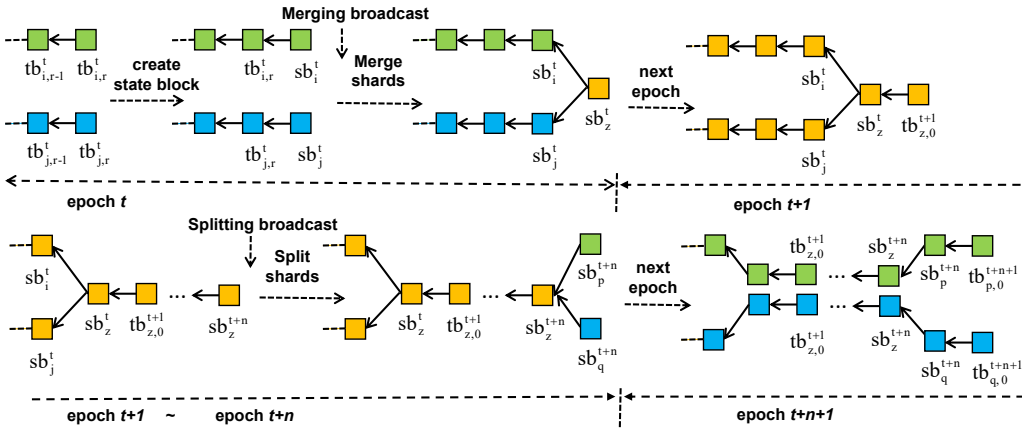


Fig. 3: The procedure of merging and splitting ledgers

by the shard. On the other hand, state blocks can be used as the subsidiary blocks to merge or split blockchains. As Fig. 3 shows, at the end of epoch t, the new reconfiguration shard (i.e., yellow blockchain z) generates a new state block sb_z^t, which connects with previous state blocks of different shards, i.e., green blockchain and blue blockchain. The state block sb_z^t gathers the information of sb_i^t and sb_j^t. Then both nodes in shard S_i and shard S_j can share the ledger of each shard when they are merged into a new shard S_z at next epoch. Similarly, when a shard is split into several shards at the end of resharding stage of epoch $t+n$ (see the below of Fig. 3), the state block sb_z^{t+n} is also split into several disjoint state blocks, denoted as sb_p^{t+n} for shard S_p and sb_q^{t+n} for shard S_q of next epoch $t+n+1$. Finally, every node updates its local state database for next epoch.

In contrast to transaction blocks, the creation of state blocks needs to do a *pre-process* to gather the information of valid accounts. As Fig. 3 shows, at the start of the generation of state block sb_i^t, each node of shard S_i firstly traces the transaction history from the last state block sb_i^{t-1} to the current transaction block $tb_{i,r}^t$, and rebuilds a *Merkle Patricia Trie* (MPT) [6] of state. MPT is a data structure that is used in Ethereum to maintain a mapping between the addresses of accounts and account states. The root of this structure is only dependent on all internal data, which means that nodes will construct the same root if they maintain the same information of valid accounts. Then, each node in shard S_i respectively computes a hash value of the MPT root H_j based on each node's local view of the ledger, and sends this value to the shard's leader.

After that, the leader can obtain the current shard size m_i^t by counting the amount of hash value messages. Furthermore, there are two conditions:

1) If the leader receives more than $m_i^t f$ hash value of the same (denoted as H), it creates and broadcasts state block sb_i^t with a proof that contains enough nodes' signatures. The header of the state block $sb_i^{t,H}$ comprises shard size m_i^t, shard id i, current epoch t and the hash value of MPT root H, while the body $sb_i^{t,B}$ stores the mapping between the address of an account and account states. The nodes in shard S_i verify the block header and gossip $sb_i^{t,H}$ again with the tag $accept$ if H is equal to their local hash value. Finally, if an honest node receives at least $m_i^t f + 1$ $accept$ messages and block headers of the same, it accepts state block sb_i^t. This period is similar to the *commit* phase of a PBFT-based consensus.

2) If the leader cannot receive more than $m_i^t f$ hash value of the same within a bound of time, it will independently create a new state block with its local hash value and account database. The new state block will be added into blockchain by executing a complete intra-shard consensus.

6.2 Shards Adjustment

After completing the state update period, the system starts adjusting shards. The above DRL-based sharding algorithm is applied into this stage. The result of shards adjustment is to obtain new shards for next epoch to process transactions. The shards adjustment is designed for the following goals in mind:

1) Adaptability: The system can update the sharding strategy in term of the current environment state of sharding system.

2) Decentralization: The selection of sharding strategy does not rely on any trusted third parties.

3) Balance and efficiency: The number of nodes in each shard should be adjusted as balanced as possible with low communication overhead.

To obtain an optimum sharding strategy for next epoch t, we can utilize a designated trained DRL agent to calculate an action a_t based on the current environment state, i.e., the amount of pending transaction, the total number of nodes and the average computing power. However, this method will cause some security problems due to the centralization. To solve this problem, we introduce a distributed deployment mechanism run by the identity chain and apply it to the consensus layer of the sharding system.

Consensus on the sharding strategy. A new sharding strategy will be decided by the identity shard via an instance of intra-shard consensus. Specifically, at the start of epoch t, the identity shard takes as inputs the current environmental state $[q_{tx}, n, p_c]_t$ to compute a sharding strategy a_t, using a_t and the information of new nodes to construct a new identity block ib^t. We left it open to choose the mechanism for distributed online data collection. A representative mechanism is used by [57], where each node monitors its local context and exchanges it with others. The identity shard then runs an instance of intra-shard consensus to agree on ib^t.

After all system nodes receive the new sharding strategy for next epoch t from the new identity block, a shard

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

needs to look for other shards to merge if the new number of shards k^t decreases compared to k^{t-1}, or split spontaneously into several new shards if the new number of shards k^t increases. To help nodes merge or split shards more efficiently and evenly, we adopt a *full binary tree* structure to distribute nodes into different shards based on their identities. Therefore, the number of shards in our sharding system is always a power of 2.

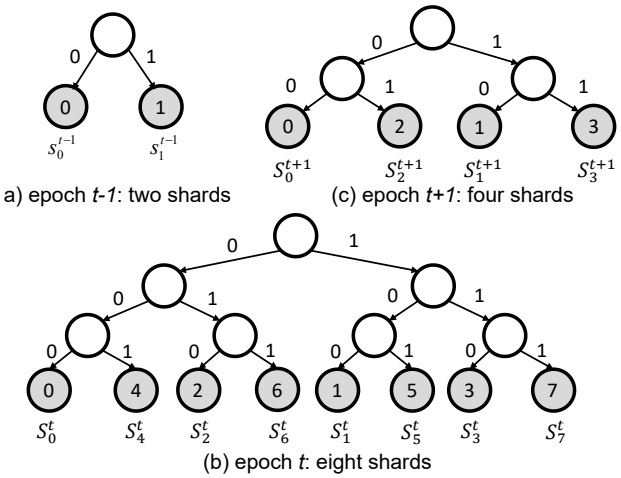


Fig. 4: Shards adjustment

A representation of the tree can be seen in Fig. 4, in which all shards are represented by the leaves of the tree. (a) At epoch $t-1$, there are only $k^{t-1}=2$ shards in the system, then the last bit of the PoW solution (i.e., the identity of a node) specifies which shard a node belongs to. After a new identity block has been agreed, every shard knows the amount of shards k^t at epoch t from the sharding strategy. (b) Shards start adjusting: A shard splits into four shards, and ultimately, there are $k^t=8$ shards at epoch t. The last 3 (log8) bits of the identity are used to specify the shard id at this epoch. (c) Similarly, when the amount of shards decreases at next epoch $t+1$, shards can merge decidedly owing to the advantages of a tree structure. For example, as Fig. 4(c) shows, the shard S_0^t will merge with its sibling shard S_4^t to form a new shard S_0^{t+1}.

6.3 Shards Reorganization

The target of shards reorganization is for nodes to establish connections with others to form new shards for transaction consensus at the new epoch. To establish point-to-point connections with their shard peers, a naive solution is to broadcast the new identity block to all nodes. However, this solution results in $O(n)$ communications and becomes inefficient when n increases. We adopt a more efficient method that only needs $O(m^t)$ complexity of communication, where m^t is the shard size in epoch t. Our algorithm to reorganize shards is depicted in Algorithm 1.

After entering shards adjustment period, the identity shard first generates a new identity block ib^t (line 2) based on DRL agent as described in Section 6.2. Then, the identity shard finds the new leaders (labeled as $newLS$) in all current shards (line 3) and broadcasts ib^t to $newLS$ (line 4). To elect a group of leaders randomly, we calculate the hashes of the randomness of the next epoch Ran_t and nodes' identities for each shard k (lines 10 to 21). A node

Algorithm 1: Algorithm to reorganize new shards for next epoch t

Input: q_{tx}^t: number of pending transactions; n^t: number of nodes; p_c^t: average computing power
Output: new shards

1 **Procedure** *ShardsOrganization*(q_{tx}^t, n^t, p_c^t)**:**
2 $ib^t \leftarrow$ IBGeneration(q_{tx}^t, n^t, p_c^t)
3 $newLS \leftarrow$ LeadersElection(Ran_t)
4 Send ib^t to $newLS$
5 **for** $l \in newLS$ **do**
6 $leaderInfo \leftarrow$ GetPeerInfo(l)
7 **for** $p \in peers$ **do**
8 l.SendInfo($p, leaderInfo, ib^t$)
9 ConnectionBuild()
10 **Procedure** *LeadersElection*(Ran_t)**:**
11 $newLS \leftarrow$ empty
12 **for** $k = 1$ *to* k^t **do**
13 $Leader_k \leftarrow n_{k,1}$
14 $Hash_k \leftarrow hash(Ran_t || ID(Leader_k))$
15 **for** $i = 2$ *to* $|S_k^t|$ **do**
16 **if** $hash(Ran_t || ID(n_{k,i})) < Hash_k$ **then**
17 $Leader_k \leftarrow n_{k,i}$
18 $Hash_k \leftarrow hash(Ran_t || ID(Leader_k))$
19 $newLS$.append($Leader_k$)
20 return $newLS$

with the minimum hash value will become the leader of its shard (line 18). Next, each of the new leaders in $newLS$ respectively informs its shard peers with ib^t and a verification $leaderInfo$ (lines 5 to 8). All nodes can quickly obtain peers' information of the new shard via new leaders, and then build connections with them (line 9). Finally, new shards have been formed and the sharding system will enter next epoch.

7 Security Analysis

In this section, we provide a formal security analysis of our dynamic sharding model. Note that the security analysis of the consensus stage is discussed in Section 4.3, and our model is exclusively designed to prevent these attacks. In the following, we analyze the security of our resharding mechanism, including state update security, shards adjustment security, and shards reorganization security.

7.1 State Update Security

Definition 5 (State update security)**.** *The state update period of epoch $t-1$ can be considered secure if honest nodes of any shard can eventually reach a validly consistent ledger state before the next epoch t.*

The state update period generates the state block (i.e., state *checkpoint*) for each shard via the intra-shard consensus. In our model, however, we cannot always guarantee that an honest leader will be selected in each shard. A malicious

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

leader can take some behaviors to lead to a failure of the state update the shard nodes cannot agree on a valid state checkpoint. We now discuss how our dynamic sharding model can ensure the state update security.

Theorem 1. *The state update security can be guaranteed if none of shards are controlled by the adversary.*

Proof. To make the state update fail, a malicious node can become non-responsive, deliver inconsistent messages, create invalid state blocks, etc. However, when the majority of a shard is honest, any malicious behaviors performed by the malicious leader can be detected, and trigger the view-change mechanism where the malicious leader is replaced by another shard node. If the new leader is honest, then the state update period could be securely taken over and the shard could eventually reach a consistent state. Nevertheless, if the new leader is malicious, the view changes will be repeated until an honest node is selected as the leader. Thus, in this case, whether shards can securely complete the state update period depends on whether an honest node is selected as the leader in this epoch.

Recall from the constraint of shard liveness (Eq. (13) and (14)), at least one honest leader can be selected if the epoch contains more than η intra-shard consensus rounds, where $\eta > \log_s 10^{-5}$. Since our dynamic sharding model is designed to satisfy the constraint of shard liveness, it guarantees that at least one honest node becomes the leader at each epoch. As a result, as long as shards are controlled by the honest nodes, our dynamic sharding system can guarantee security in the state update period. □

7.2 Shards Adjustment Security

Definition 6 (Shards adjustment security)**.** *The shards adjustment period of epoch $t-1$ is said to be secure if the sharding strategy is made by honest nodes.*

Theorem 2. *The shards adjustment security is guaranteed in our sharding protocol.*

Proof. Recall from Section 6.2 that the sharding strategy is made by the identity shard via an instance of intra-shard consensus. As the system guarantees that each shard contains fewer than $1/3$ Byzantine nodes (Eq. (16)), the intra-shard consensus can be completed and must be voted on by at least $f+1$ honest nodes. Therefore, the shards adjustment security is guaranteed. □

7.3 Shards Reorganization Security

Definition 7 (Shards adjustment security)**.** *The shards reorganization period of epoch $t-1$ is said to be secure if honest nodes of each shard can establish reliable connections and the consensuses of epoch t are dominated by honest nodes.*

In the shards reorganization period, we appoint shard leaders to assist in establishing connections among shard nodes. However, a dishonest leader can construct invalid connection information and provide honest nodes with these information to control the communications of the shard. After that, the attackers can launch a number of network layer attacks to control consensus process [58], [59].

Theorem 3. *The shards reorganization security can be ensured if the system fault tolerance is no more than $1/4$.*

Proof. To prove Theorem 3, we need to prove every shard cannot deviate from the shards reorganization, i.e., the shard leader (honest or malicious) cannot combine with other malicious nodes to control connections among honest nodes of the shard. Obviously, the total number of malicious nodes in a shard will influence the probability that the adversary can take over the shards reorganization period. Therefore, to compute the bounds of relevant system parameters under the system model and security assumptions defined in Section 3, we consider the most vulnerable shard that has the highest number of malicious nodes among all other shards.

Without loss of generality, we assume that there are $\vartheta_v^{t-1} = f_{max} m^{t-1}$ malicious nodes in such a vulnerable shard S_v^{t-1} at epoch $t-1$, where m^{t-1} is the shard size and f_{max} is the maximum proportion of malicious nodes that the adversary can be assigned in one shard. Since our dynamic sharding model allows shards to be merged or split in the shards reorganization period, the number of malicious nodes will change in a new shard. Specifically, we use x^t as the change factor of shards number, i.e., every x^t shards are merged into one shard when the number of shards k^t reduces, or each shard is split into $\frac{1}{x^t}$ shards when k^t increases at epoch t. Moreover, considering there will be y^t new nodes participating in the system or $|y^t|(y^t < 0)$ old nodes leaving the system at next epoch t, we can eventually calculate the number of malicious nodes ϑ^t in a new shard S_i^t at epoch t satisfies the following formula:

$$\begin{aligned} \vartheta_v^t &= \sum_{k=1}^{x^t} f_{v,k} m^{t-1} + x^t s_{v,k} y^t \\ &\leq f_{max} x^t m^{t-1} + x^t s y^t, \end{aligned} \tag{21}$$

where $f_{v,k} (\leq f_{max})$ denotes the proportion of malicious nodes in shard $S_{v,k}^{t-1}$, and $s_{v,k}$ is the proportion of malicious nodes of joining or leaving nodes. Recall from our system model and security assumptions (Section 3), the system fault tolerance s is assumed to be always satisfied no matter how many nodes join or leave the system, and thus $s_{v,k}$ should be less than s. Moreover, since f_{max} is assumed as the maximum proportion of malicious nodes at epoch $t-1$, $f_{v,k}$ should be less than f_{max}. Finally, the below of Eq. (21) represents the maximum number of malicious nodes in a shard at epoch t. Similarly, the number of nodes m^t in a new shard at epoch t can be evaluated by

$$m^t = x^t (m^{t-1} + y^t). \tag{22}$$

To compromise the security of the shards reorganization period, the adversary in the new vulnerable shard S_v^t needs to take over the shard. If the shard leader is honest, then the security can be ensured as long as the proportion of malicious nodes in S_v^t is less than the shard fault tolerance, i.e., $f_{max} \leq f = 1/3$ (see Section 3.2). However, when the shard leader is malicious, an honest node may receive a group of invalid nodes' information (e.g., repeated IPs or unregistered IPs) provided by the malicious leader. We assume that the malicious shard leader provides ϖ honest nodes with invalid information while the left honest nodes

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

can receive valid information. There are two conditions as follows:

1) $\varpi > \vartheta_v^t$: Since the valid information of nodes is recorded on the identity blockchain, a node can verify the validity of $peerInfo$ provided by the new shard leader. As a result, ϖ nodes can detect malicious behavior and broadcast it with a *reject* message to other nodes. Since at most ϑ_v^t malicious nodes exist in a shard, the malicious leader will be detected by other nodes once they receive more than ϑ_v^t reject messages. Finally, the malicious leader is rejected and replaced by re-electing a new node to be the leader of the new shard.

2) $\varpi \leq \vartheta_v^t$: In this case, the ϖ honest nodes reject the leader by sending *reject* messages along with view-change requests, and cannot take part in the next consensus until they find the shard can still work well at next epoch in which these ϖ honest nodes will re-establish connections with shard nodes. However, this will not trigger a view-change to replace the malicious shard node before the shard enters epoch t, since there are not enough nodes ($< \vartheta_v^t$) to propose the view-change request. In this case, the residue of honest nodes χ is

$$\begin{aligned}\chi &= m^t - \varpi - \vartheta_v^t \\ &\geq m^t - \vartheta^t - \vartheta_v^t \\ &\geq x^t(m^{t-1} + y^t - 2f_{max}m^{t-1} - 2sy^t).\end{aligned} \tag{23}$$

Since these ϖ honest nodes that received invalid messages refuse temporarily to participate in the next consensus, the number of left nodes (i.e., $m^t - \varpi$) will be the consensus group in the following consensus rounds. To ensure the security of the shards reorganization that the adversary cannot take over the consensus group, the proportion of malicious nodes should satisfy the fault tolerance of the consensus group, i.e., $1/3$ in our PBFT-based intra-shard consensus. Specifically, χ should satisfy

$$\begin{aligned}\chi &\geq \frac{2}{3}(m^t - \varpi) \\ &\geq \frac{2}{3}x^t(m^{t-1} + y^t - f_{max}m^{t-1} - sy^t).\end{aligned} \tag{24}$$

From Eq. (23) and (24), we can conclude the following formula:

$$m^{t-1} + y^t \geq 4f_{max}m^{t-1} + 4sy^t. \tag{25}$$

Note that the maximum proportion of malicious nodes in a shard must not be smaller than the system fault tolerance (i.e., $f_{max} \geq s$); otherwise, the shard fault tolerance f will be smaller than the system fault tolerance s since the total number of malicious nodes in the system is equal to the number of malicious nodes in each shard, which contradicts the security assumption illustrated in Section 3.2. Based on this rule, the Eq. (25) can be further derived as: $m^{t-1} + y^t \geq 4s(m^{t-1} + y^t)$. Eventually, it can be concluded that $s \leq 1/4$, which indicates the bound of the relevant system parameter that the security of the shards reorganization is ensured. □

TABLE 2: Evaluation Parameters

Parameter	Value
The original number of nodes, n^0	8000
The original number of transactions, q_{tx}^0	2×10^5
The fault tolerance of the system, s	$\frac{1}{4}$
The fault tolerance of a shard, f	$\frac{1}{3}$
The security parameter, λ	5
Attack ability per attack, ϵ	1.6×10^{-6}
The participating constraint factor, $\varkappa$	$\frac{1}{2}$
The cost for randomness creation, κ	20s
The evaluation factor for shard adjustment, α	10
The cost for reshuffled nodes, β	200
The latency of validation, t_v	1s
The cost for adding new block, t_a	2s
The transmission rate of data, R_t	1Mbps
Average size of block header, S^H	548B
Average size of transactions, S^T	512B

8 EVALUATION

8.1 Setup

We develop a simulation testbed with 8,000 initial validators and 16 shards in Python[2]. In the testbed, at each epoch, every shard chooses a new leader based on randomness. The leaders of all shards respectively pack transactions into new blocks and then complete consensus via PBFT protocol. The overhead of a consensus is evaluated by Eq. (1). To simulate the dynamic environment of blockchain systems, we assume that the arrivals of transactions and the leaving of validators follow a Poisson process. In addition, we assume that a number of new validators participate in the system at every epoch, and each node has an equal computing power, which means that each new node has the same opportunity to join the system. The implementation of Byzantine behaviors is considered an open problem [60]. For simplicity, we simulate and model the node corruption as a random process proportional to n, where honest nodes are randomly corrupted. That is, honest nodes are randomly chosen to be corrupted, but it takes a while (proportional to n, s, and ϵ as we formulate in Section 4.3) to fully corrupt the target honest nodes. For the security parameters, we set them unchanged (as listed in Table 2) unless we discuss them explicitly.

We use Proximal Policy Optimization (PPO) [61] algorithm as our DRL-based sharding model, which is an efficient policy gradient method that achieves a favorable balance between sample complexity, simplicity, and wall-time. The PPO model is implemented in improved OpenAI baselines [3]. The proposed DRL-based sharding model is trained in Windows Server 2016 with Intel(R) Xeon(R) CPU E5-2650 v4 and 64GB RAM. The parameter settings of the sharding system used in simulations are outlined in Table 2. Since we use transaction throughput to evaluate reward,

2. https://www.python.org/
3. https://github.com/openai/baselines/tree/master/baselines/ppo2

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

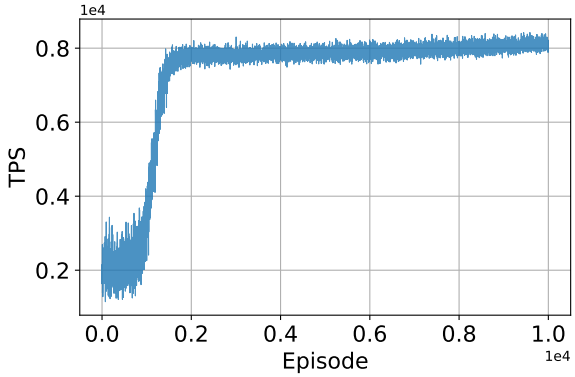


Fig. 5: Convergence performance.

and set security formulations as constraint conditions of the reward function, higher cumulative rewards represent a better trade-off between system performance and security. Besides, to speed up the convergence, we set a boundary of sharding strategies, where the consensus rounds r is in the range of $(0, 200)$, the maximum amount of shards is 256, the block size is limited within 1MB to 10MB and the PoW difficulty is set as an integer less than 100. The designs of sharding algorithms for this paper are listed as follows:

1) *Improved DRL-based dynamic sharding algorithm (DBDSA+)*. The dynamic sharding algorithm proposed in Section 5.2, which is trained with PPO model.

2) *DRL-based dynamic sharding algorithm (DBDSA)*. The dynamic sharding algorithm proposed in our previous work [1], which was trained with a deep deterministic policy gradient algorithm and did not consider the change of average computing power, as well as the overhead introduced by the resharding protocol. We retest this algorithm with the parameter setting in Table 2.

3) *Greedy dynamic sharding algorithm (GDSA)*. The idea of a greedy dynamic sharding algorithm selects a sharding strategy that can obtain a maximum reward in the current epoch, and does not consider the future influence of the sharding strategy.

4) *Static sharding algorithm (SSA)*. The static sharding algorithm is adopted in most previous works, e.g., OmniLedger [21] and RapidChain [22]. In these static sharding systems, the sharding strategy (i.e., the consensus rounds, the number of shards, the block size, and the PoW difficulty) is fixed all the time regardless of the changes environment in the sharding system. Specifically, we set the consensus rounds $r = 100$, the number of shards $k = 32$, the block size $S^B = 4MB$, and the PoW difficulty $p_d = 10$.

8.2 Convergence

We set 100 steps as an episode, in which the system makes 100 sharding strategies to calculate the cumulative rewards based on the reward function Eq. (20) (i.e., the average transaction throughput). The cumulative rewards change curve in the training process of DBDSA+ is shown in Fig. 5. In Fig. 5, the y-axis shows the average TPS of each 100 steps while the x-axis is the training episode. Note that the reward of a sharding strategy is set to be 0 when the sharding strategy does not satisfy security constraints as Eq. (19) describes. It can be seen that the cumulative rewards are increasing obviously and rapidly in approximately 1,000 episodes, after the DRL agent has learnt a performance-friendly and security-friendly sharding strategy. After 2,000 episodes of training, the model basically reaches convergence. From Fig. 5, we can conclude that: (1) DBDSA+ can quickly adapt to the dynamic environment, which is crucial for a real-time dynamic blockchain sharding system to adjust the sharding strategy. (2) DBDSA+ can address our optimization problem because it can reach a final convergence.

8.3 Performance

Fig. 6 shows some comparisons among four sharding algorithms in throughput. The throughput, which is used to measure performance, is calculated by the reward function Eq. (20). From these figures, we can find that our proposed DRL-based sharding algorithms (i.e., the DBDSA+ and the DBDSA) outperform other non-DRL based baselines, which indicates that DRL-based sharding algorithms are more adaptive to make a sharding policy under the dynamic environment of sharding systems. Besides, the DBDSA+ obtains higher throughput than the DBDSA, since the former takes into consideration the dynamics of average computing power and adjusts the PoW difficulty to resist the multiple identities attack.

To show that our proposed DRL-based sharding algorithm can maintain a long-term trade-off between system performance and security, we set different numbers of sharding steps to compare the cumulative rewards ofthe above sharding algorithms. Fig. 6(a) illustrates the change of throughput with the increase number of sharding steps. From this picture, we find that the throughput of the GDSA is fluctuating with the increase of sharding steps, while other algorithms remain steady. This is because the GDSA only pursues the maximum current reward, without considering the change of environment in the future. To pursue higher throughput, the greedy dynamic sharding algorithm will set more consensus rounds and a small shard size in terms of the current environment under the promise of security constraints. But it also provides more time and requires less number of honest validators (i.e., 1/3 shard size) for the malicious nodes to corrupt the system. If there are too many validators leaving a shard before the next resharding stage, the system will become unsafe, and the reward will drop to 0.

The effect of attack ability ϵ for performance is illustrated in Fig. 6(b). According to Eq. (11), the time required for corruption will shorten with the improvement of attack ability. To deal with the improvement of attack ability, a sharding system should shorten the resharding interval (i.e., decrease the consensus rounds) or enlarge the shard size. From Fig. 6(b), we can find that the system performance of the greedy dynamic sharding algorithm degrades obviously compared with other sharding algorithms with the increase of attack ability. Furthermore, our proposed DBDSA+ has the best performance while SSA has a poor performance all the time. The performance with the change of security parameter λ is shown in Fig. 6(c). From Eq. (9) to (10), we can conclude that the larger λ will result in poorer performance. We will combine 7(a) and 7(b) to analyze the reason in Section 8.4.

In Fig. 6(d), we discuss the impact of reshuffling latency β. Reshuffling latency measures the cost that a portion of

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

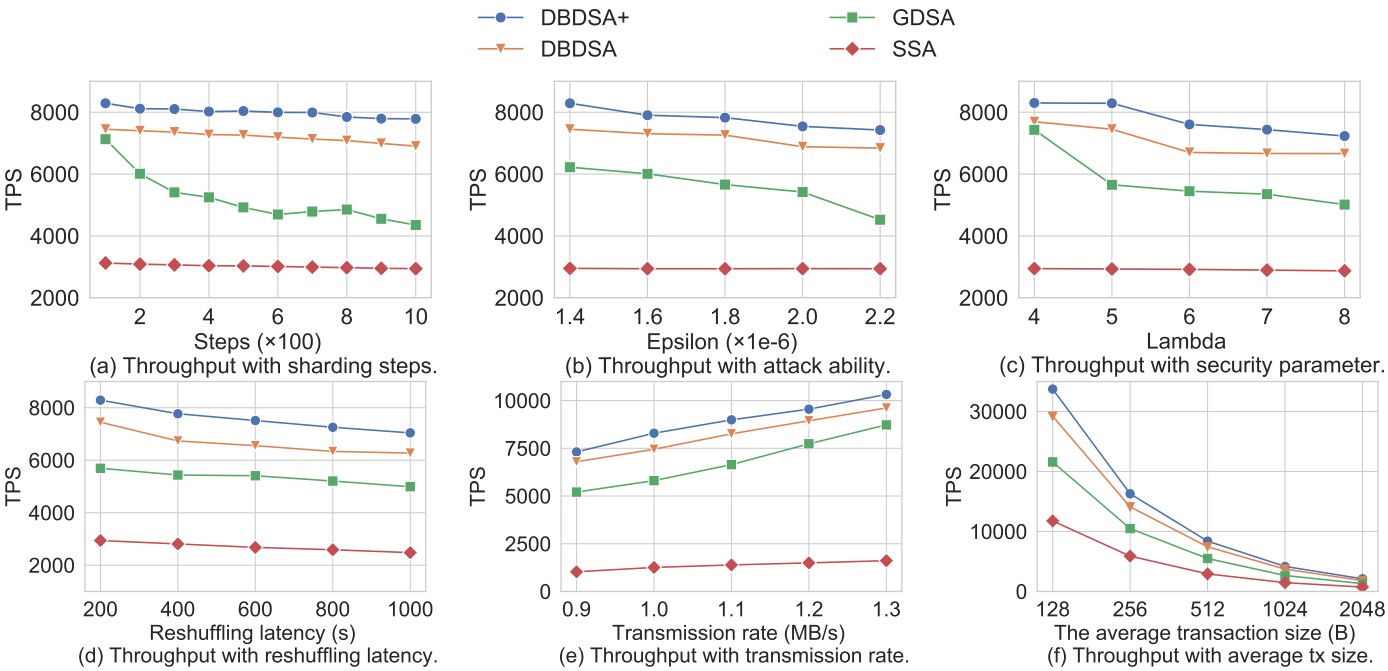


Fig. 6: Throughput with different parameters.

nodes being reshuffled from one shard to another shard during the resharding stage. From Fig. 6(d), we find that performance of sharding systems degrades with increased reshuffling latency. Therefore, although the resharding stage helps the system resist attacks, it also introduces extra overhead, which indicates the frequency of resharding should be adjusted properly to balance system performance and security of sharding.

From Fig. 6(e), we conclude that the faster data transmission rate helps enhance the system performance. Note that the dynamic sharding algorithms (i.e., DBDSA+, DBDS, and GDSA) have a more rapid increase of throughput with improvement of transmission rate than the SSA. This result shows that the transmission rate is not the bottleneck to enhance the throughput of sharding systems that adopt the static sharding strategy. To study the influence of average transaction size on throughput, we set average transaction size $S^T = 128, 256, 512, 1024, 2048$ bytes, to respectively compute the throughput with different sharding algorithms. Fig. 6(f) shows that the DBDSA+ obtains the highest throughput all the time while the SSA performs worst. Besides, we can observe that a larger average transaction size will decrease transaction throughput. This is because a block could pack less number of transactions each time, leading to lower throughput according to Eq. (8). Furthermore, compared to 7 TPS with the average transaction size (250 bytes) in Bitcoin, our proposed dynamic sharding can reach more than 10,000 TPS when the average transaction size is set to 256 bytes.

8.4 Security

Fig. 7(a) and (b) illustrate the resilience for attacks of four sharding algorithms under different conditions. The y-axis *unsafe steps* represents the cumulative times that sharding strategies selected by different sharding algorithms cannot satisfy the safe constraints (i.e., Eq. (19)). We respectively simulate these sharding algorithms in 1000 continuous

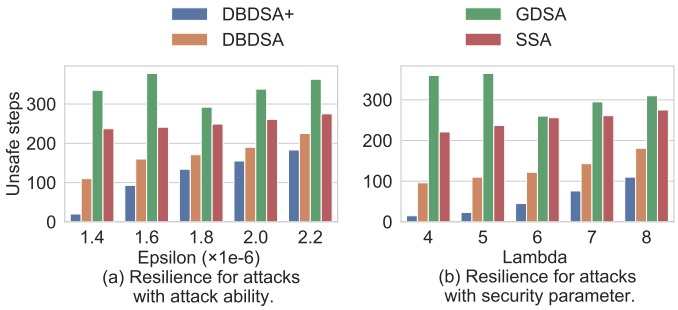


Fig. 7: The total unsafe times within 1000 sharding steps.

sharding strategies. Fig. 7(a) shows that the enhancement of the attack ability ϵ will increase the risk that a sharding system suffers from the corruption attack. Fig. 7(b) shows that the higher security parameter λ will result in the stricter safe constraint. With the increase of λ, the shard size should be larger to satisfy the safe constraint Φ_1 in Eq. (19). In other words, the amount of shards has to be reduced to guarantee shards large enough to resist the unsafe shard formation attack, as described in Section 4.3. Combining Fig. 6(b) and 6(c) with Fig. 7(a) and 7(b), we can see that the DBDSA+ has the highest throughput and the least times of breaking the safe constraints, and DBDS comes second. This result shows that our proposed DRL-based dynamic sharding algorithm can not only obtain a high throughput, but also be safer to resist malicious attacks. Besides, although the GDSA is easier to break the safe constraints than the SSA, the GDSA still obtains higher throughput than the SSA. This is because the GDSA always makes a more adventurous sharding strategy to pursue higher throughput in terms of the current environment, e.g., divides more shards, while the SSA does nothing to meet the dynamic environment. This result shows that the SSA cannot balance system performance and security in a dynamic sharding system.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

9 Conclusion and Discussion

In this paper, we focus on scaling the blockchain with an intelligently and efficiently dynamic sharding. We first build an evaluation framework to evaluate system performance and security, and conclude the optimization target. Next, we apply a DRL algorithm to address this optimization problem, i.e., making optimal sharding strategies for different environmental states of the sharding system. To reduce the overhead introduced by resharding, we propose an adaptive resharding protocol that helps to reshard the system efficiently and securely. Finally, we experiment with our model in a simulated blockchain sharding environment. Experimental results show our proposed framework can achieve 2.8 times TPS than other static sharding systems and higher security.

There are many future directions. First, our protocol relies on well-trained DRL agents to make effective sharding strategies. However, training and deploying DRL agents in a distributed setting might introduce extra overheads in terms of computation and communication. Such overheads can be reduced from two perspectives. First, the main training process is offloaded from the sharding protocol. This can be done by using some real-world data (e.g., the number of historic validators in Ethereum) to pre-train the DRL agents before deployment. Second, to reduce the overhead of deploying the DRL agents and enhance scalability, only the identity shard (rather than all shards) participates in sharding strategy decisions.

Second, when relying on the consensus protocol to reach an agreement on sharding strategies made by DRL agents, its required new properties are not well-defined and not fully explored in the current literature. The existing BFT consensus protocols are designed to exclusively maintain safety and liveness properties. When using them as a blackbox to make DRL decisions consistently, it is unclear what modifications are required to make it compatible with DRL agents. For instance, we should allow different nodes' DRL agents to have slightly distinct neural network parameters, as the nodes might have various local views on the environment.

References

[1] J. Zhang, Z. Hong, X. Qiu, Y. Zhan, S. Guo, and W. Chen, "Skychain: A deep reinforcement learning-empowered dynamic blockchain sharding system," in *49th International Conference on Parallel Processing-ICPP*, 2020, pp. 1–11.

[2] S. Ma, Y. Deng, D. He, J. Zhang, and X. Xie, "An efficient nizk scheme for privacy-preserving transactions over account-model blockchain," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 2, pp. 641–651, 2021.

[3] Y. Jiao, P. Wang, D. Niyato, and K. Suankaewmanee, "Auction mechanisms in cloud/fog computing resource allocation for public blockchain networks," *IEEE Trans. Parallel Distributed Syst.*, vol. 30, no. 9, pp. 1975–1989, 2019. [Online]. Available: https://doi.org/10.1109/TPDS.2019.2900238

[4] M. Li, J. Weng, A. Yang, W. Lu, Y. Zhang, L. Hou, J. Liu, Y. Xiang, and R. H. Deng, "Crowdbc: A blockchain-based decentralized framework for crowdsourcing," *IEEE Trans. Parallel Distributed Syst.*, vol. 30, no. 6, pp. 1251–1266, 2019. [Online]. Available: https://doi.org/10.1109/TPDS.2018.2881735

[5] S. Nakamoto *et al.*, "Bitcoin: A peer-to-peer electronic cash system," 2008.

[6] G. Wood *et al.*, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum project yellow paper*, vol. 151, no. 2014, pp. 1–32, 2014.

[7] C. Xu, K. Wang, P. Li, S. Guo, J. Luo, B. Ye, and M. Guo, "Making big data open in edges: A resource-efficient blockchain-based approach," *IEEE Transactions on Parallel and Distributed Systems*, vol. 30, no. 4, pp. 870–882, 2019.

[8] M. Zhaofeng, W. Xiaochang, D. K. Jain, H. Khan, G. Hongmin, and W. Zhen, "A blockchain-based trusted data management scheme in edge computing," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 3, pp. 2013–2021, 2019.

[9] S. Misra, A. Mukherjee, A. Roy, N. Saurabh, Y. Rahulamathavan, and M. Rajarajan, "Blockchain at the edge: Performance of resource-constrained iot networks," *IEEE Trans. Parallel Distributed Syst.*, vol. 32, no. 1, pp. 174–183, 2021. [Online]. Available: https://doi.org/10.1109/TPDS.2020.3013892

[10] K. Karlsson, W. Jiang, S. Wicker, D. Adams, E. Ma, R. van Renesse, and H. Weatherspoon, "Vegvisir: A partition-tolerant blockchain for the internet-of-things," in *2018 IEEE 38th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2018, pp. 1150–1158.

[11] E. Mengelkamp, B. Notheisen, C. Beer, D. Dauer, and C. Weinhardt, "A blockchain-based smart grid: towards sustainable local energy markets," *Computer Science-Research and Development*, vol. 33, no. 1, pp. 207–214, 2018.

[12] K. Gai, Y. Wu, L. Zhu, M. Qiu, and M. Shen, "Privacy-preserving energy trading using consortium blockchain in smart grid," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 6, pp. 3548–3558, 2019.

[13] L. N. Nguyen, T. D. Nguyen, T. N. Dinh, and M. T. Thai, "Optchain: Optimal transactions placement for scalable blockchain sharding," in *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2019, pp. 525–535.

[14] V. Buterin, "Ethereum sharding faq," Jan. 2021. [Online]. Available: https://github.com/ethereum/wiki/wiki/Sharding-FAQ

[15] G. Wang, Z. J. Shi, M. Nixon, and S. Han, "Sok: Sharding on blockchain," in *Proceedings of the 1st ACM Conference on Advances in Financial Technologies*. ACM, 2019, pp. 41–61.

[16] R. Pass and E. Shi, "Hybrid consensus: Efficient consensus in the permissionless model," in *31st International Symposium on Distributed Computing (DISC 2017)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2017.

[17] H. Team, "Harmony technical whitepaper." [Online]. Available: https://harmony.one/whitepaper.pdf

[18] J. R. Douceur, "The sybil attack," in *International workshop on peer-to-peer systems*. Springer, 2002, pp. 251–260.

[19] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A secure sharding protocol for open blockchains," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '16. New York, NY, USA: ACM, 2016, pp. 17–30.

[20] M. Al-Bassam, A. Sonnino, S. Bano, D. Hrycyszyn, and G. Danezis, "Chainspace: A sharded smart contracts platform," *arXiv preprint arXiv:1708.03778*, 2017.

[21] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "Omniledger: A secure, scale-out, decentralized ledger via sharding," in *2018 IEEE Symposium on Security and Privacy (SP)*, May 2018, pp. 583–598.

[22] M. Zamani, M. Movahedi, and M. Raykova, "Rapidchain: Scaling blockchain via full sharding," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '18. New York, NY, USA: ACM, 2018, pp. 931–948.

[23] J. Wang and H. Wang, "Monoxide: Scale out blockchains with asynchronous consensus zones," in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. Boston, MA: USENIX Association, Feb. 2019, pp. 95–112.

[24] T. Z. Team, "The zilliqa technical whitepaper," Aug 2017. [Online]. Available: https://harmony.one/whitepaper.pdf

[25] H. Dang, T. T. A. Dinh, D. Loghin, E.-C. Chang, Q. Lin, and B. C. Ooi, "Towards scaling blockchain systems via sharding," in *Proceedings of the 2019 International Conference on Management of Data*. ACM, 2019, pp. 123–140.

[26] Y. Liu, J. Liu, Q. Wu, H. Yu, H. Yiming, and Z. Zhou, "Sshc: A secure and scalable hybrid consensus protocol for sharding blockchains with a formal security framework," *IEEE Transactions on Dependable and Secure Computing*, 2020.

[27] Z. Hong, S. Guo, P. Li, and W. Chen, "Pyramid: A layered sharding blockchain system," in *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications*, 2021.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057

[28] M. Castro and B. Liskov, "Practical byzantine fault tolerance," in *Proceedings of the Third Symposium on Operating Systems Design and Implementation*, ser. OSDI '99. USA: USENIX Association, 1999, p. 173–186.

[29] V. King and J. Saia, "Breaking the o (n 2) bit barrier: scalable byzantine agreement with an adaptive adversary," *Journal of the ACM (JACM)*, vol. 58, no. 4, pp. 1–24, 2011.

[30] B. Awerbuch and C. Scheideler, "Towards a scalable and robust dht," *Theory of Computing Systems*, vol. 45, no. 2, pp. 234–260, 2009.

[31] S. Sen and M. J. Freedman, "Commensal cuckoo: Secure group partitioning for large-scale services," *SIGOPS Oper. Syst. Rev.*, vol. 46, no. 1, pp. 33–39, Feb. 2012.

[32] M. Li, X. Luo, K. Xue, Y. Xue, W. Sun, and J. Li, "A secure and efficient blockchain sharding scheme via hybrid consensus and dynamic management," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 5911–5924, 2024.

[33] A. Liu, J. Chen, K. He, R. Du, J. Xu, C. Wu, Y. Feng, T. Li, and J. Ma, "Dynashard: Secure and adaptive blockchain sharding protocol with hybrid consensus and dynamic shard management," *IEEE Internet of Things Journal*, 2024.

[34] T. Li, Y. Wu, J. Weng, J. Lu, S. Huang, and W. Deng, "Abchain: A dynamic sharding protocol with balanced account partitioning and adaptive brokers for iot blockchain," *IEEE Internet of Things Journal*, 2025.

[35] H. Dai, L. Yuan, J. Wu, H. Chen, and H. Bao, "Dbsrp-ml: Dynamic blockchain sharding reconfiguration protocol based on multi-label," *Computer Networks*, p. 111401, 2025.

[36] K. Khacef, S. Benbernou, M. Ouziri, and M. Younas, "A dynamic sharding model aware security and scalability in blockchain," *Information Systems Frontiers*, vol. 26, no. 6, pp. 2323–2336, 2024.

[37] Y. Wang, Z. Gong, D. Jia, A. Tan, and M. Liu, "Dynamic sharding model and performance optimization method for consortium blockchain," *The Journal of Supercomputing*, vol. 81, no. 2, p. 411, 2025.

[38] B. David, B. Magri, C. Matt, J. B. Nielsen, and D. Tschudi, "Gearbox: Optimal-size shard committees by leveraging the safety-liveness dichotomy," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 683–696.

[39] J. Zhang, Z. Luo, R. Ramesh, and A. Kate, "Optimal sharding for scalable blockchains with deconstructed smr," *VLDB*, 2025.

[40] Z. Hong, S. Guo, E. Zhou, J. Zhang, W. Chen, J. Liang, J. Zhang, and A. Zomaya, "Prophet: Conflict-free sharding blockchain via byzantine-tolerant deterministic ordering," in *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 2023, pp. 1–10.

[41] J. Zhang, W. Chen, S. Luo, T. Gong, Z. Hong, and A. Kate, "Front-running attack in sharded blockchains and fair cross-shard consensus," *NDSS*, 2024.

[42] Y. Liu, A. Liu, Y. Lu, Z. Pan, Y. Li, J. Liu, S. Bian, and M. Conti, "Kronos: A secure and generic sharding blockchain consensus with optimized overhead," *NDSS*, 2025.

[43] T. Cai, W. Chen, J. Zhang, and Z. Zheng, "Smartchain: A dynamic and self-adaptive sharding framework for iot blockchain," *IEEE Transactions on Services Computing*, vol. 17, no. 2, pp. 674–688, 2024.

[44] M. Liu, F. R. Yu, Y. Teng, V. C. Leung, and M. Song, "Performance optimization for blockchain-enabled industrial internet of things (iiot) systems: A deep reinforcement learning approach," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 6, pp. 3559–3570, 2019.

[45] C. Qiu, H. Yao, F. R. Yu, C. Jiang, and S. Guo, "A service-oriented permissioned blockchain for the internet of things," *IEEE Transactions on Services Computing*, vol. 13, no. 2, pp. 203–215, 2019.

[46] J. Yun, Y. Goh, and J.-M. Chung, "Dqn-based optimization framework for secure sharded blockchain systems," *IEEE Internet of Things Journal*, vol. 8, no. 2, pp. 708–722, 2020.

[47] R. Neiheiser, M. Matos, and L. Rodrigues, "Kauri: Scalable bft consensus with pipelined tree-based dissemination and aggregation," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, 2021, pp. 35–48.

[48] J. Hellings and M. Sadoghi, "Byshard: Sharding in a byzantine environment," *Proceedings of the VLDB Endowment*, vol. 14, no. 11, pp. 2230–2243, 2021.

[49] M. J. Amiri, D. Agrawal, and A. El Abbadi, "Sharper: Sharding permissioned blockchains over network clusters," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 76–88.

[50] S. Das, V. Krishnan, I. M. Isaac, and L. Ren, "Spurt: Scalable distributed randomness beacon with transparent setup," in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 2502–2517.

[51] Y. Yang, B. Li, Q. Wu, B. Qin, Q. Wang, S. Xiong, and W. Susilo, "Randflash: Breaking the quadratic barrier in large-scale distributed randomness beacons," *IEEE Transactions on Information Forensics and Security*, 2025.

[52] A. Bandarupalli, A. Bhat, S. Bagchi, A. Kate, and M. K. Reiter, "Random beacons in monte carlo: Efficient asynchronous random beacon without threshold cryptography," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 2621–2635.

[53] Y. Xu, J. Zheng, B. Düdder, T. Slaats, and Y. Zhou, "A two-layer blockchain sharding protocol leveraging safety and liveness for enhanced performance," *NDSS*, 2024.

[54] A. Kiayias, A. Russell, B. David, and R. Oliynykov, "Ouroboros: A provably secure proof-of-stake blockchain protocol," in *Annual International Cryptology Conference*. Springer, 2017, pp. 357–388.

[55] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich, "Algorand: Scaling byzantine agreements for cryptocurrencies," in *Proceedings of the 26th Symposium on Operating Systems Principles*, 2017, pp. 51–68.

[56] M. Castro and B. Liskov, "Practical byzantine fault tolerance and proactive recovery," *ACM Transactions on Computer Systems (TOCS)*, vol. 20, no. 4, pp. 398–461, 2002.

[57] C. Wu, H. Qin, M. J. Amiri, B. T. Loo, D. Malkhi, and R. Marcus, "{BFTBrain}: Adaptive {BFT} consensus with reinforcement learning," in *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, 2025, pp. 1563–1583.

[58] E. Heilman, A. Kendler, A. Zohar, and S. Goldberg, "Eclipse attacks on bitcoin's peer-to-peer network," in *Proceedings of the 24th USENIX Conference on Security Symposium*, 2015, pp. 129–144.

[59] M. Tran, I. Choi, G. J. Moon, A. V. Vu, and M. S. Kang, "A stealthier partitioning attack against bitcoin peer-to-peer network," in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 894–909.

[60] S. Bano, A. Sonnino, A. Chursin, D. Perelman, Z. Li, A. Ching, and D. Malkhi, "Twins: Bft systems made robust," in *25th International Conference on Principles of Distributed Systems*, 2022.

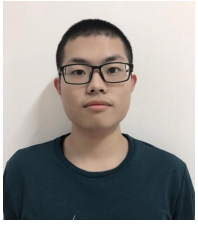
[61] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.



Jianting Zhang received his bachelor's and master's degrees from Sun Yat-Sen University, China, in 2019 and 2022. He will pursue his Ph.D. degree at Purdue University in August 2022. His research focuses on blockchain, smart contract, system security, and distributed consensus algorithms.



Zicong Hong is currently pursuing the Ph.D degree in Department of Computing in The Hong Kong Polytechnic University. He received his B.Eng. degree in software engineering with the School of Data and Computer Science, Sun Yat-sen University. His current research interest includes Blockchain, Game Theory, Internet of Things, and Edge/Cloud Computing.



Xiaoyu Qiu received the B.S. degree from Sun Yat-Sen University, Guangzhou, China, in 2020. He is currently pursuing the M.S. degree at the School of Data and Computer Science, Sun Yat-Sen University, Guangzhou, China. He is proactively working on edge computing, cloud computing, cloud robotics and computation offloading, with emphasis on artificial intelligence in edge/cloud computing.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

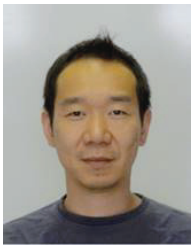
This article has been accepted for publication in IEEE Transactions on Dependable and Secure Computing. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TDSC.2025.3621057



Wuhui Chen is an associate professor in Sun Yat-sen University, China. He received his bachelor's degree from Northeast University, China, in 2008. He received his master's and Ph.D. degrees from University of Aizu, Japan, in 2011 and 2014, respectively. From 2014 to 2016, he was a JSPS research fellow in Japan. From 2016 to 2017, he was a researcher in University of Aizu, Japan. His research interests include Edge/Cloud Computing, Cloud Robotics, and Blockchain.



Yufeng Zhan received his Ph.D. degree in the School of Automation from Beijing Institute of Technology, China, in 2018. He is an Assistant Professor in the Beijing Institute of Technology. Prior to join BIT, he was a post-doctoral fellow in the Department of Computing with The Hong Kong Polytechnic University. His research interests include networking systems, game theory, and machine learning.



Song Guo received his Ph.D. in computer science from the University of Ottawa. He is currently a full professor in the Department of Computing with The Hong Kong Polytechnic University. Prior to joining PolyU, he was a full professor with the University of Aizu, Japan. His research interests are mainly in the areas of cloud and green computing, big data, wireless networks, and cyber-physical systems. He has published over 300 conference and journal papers in these areas and received multiple best paper awards from IEEE/ACM conferences. His research has been sponsored by JSPS, JST, MIC, NSF, NSFC, and industrial companies. He has served as an editor for several journals, including IEEE TPDS, IEEE TETC, IEEE TGCN, IEEE Communications Magazine, and Wireless Networks. He has been actively participating in international conferences as general chair and TPC chair. He is a Fellow of IEEE, a senior member of ACM, and an IEEE Communications Society Distinguished Lecturer.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 08:23:41 UTC from IEEE Xplore. Restrictions apply.
© 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted,
but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.